

DATA9001

FUNDAMENTALS OF DATA SCIENCE

Apple Quality Classification using KNN

For this problem, it is asked to find the quality of apples based on various features using the K Nearest Neighbour (KNN) method. The data set contained features such as size, weight, sweetness, crunchiness, juiciness, ripeness, acidity, and the target variable, which indicates the overall quality of the fruit. In order to solve the problem, the following steps were taken:

- I. Data Preprocessing
- II. Data Splitting
- III. Model Implementation
- IV. Model Evaluation

The data is cleaned, pre-processed and trained to evaluate the classification accuracy, precision, recall and F1 score to analyse the insights into the given characteristics.

I. Data Preprocessing

By processing data, we make sure that the given data set contains all features on a similar scale by cleaning and normalising the data. The following steps were taken to pre-process the data to make it ready for model training and evaluation:

- a. Replaced invalid entries with NaN
- b. Converted features to numeric for ease of calculation
- c. Filled 'NaN' values with the median of each column
- d. Standardized 'Quality' column
- e. Deleted rows with NaN values in 'Quality' column
- f. Removed outliers using Z-score method

```
df['Quality'].unique()

array(['bad', 'good', 'GOOD', 'BAD', nan, 'invalid', 'error', '-9999',
      '9999'], dtype=object)

# Replace invalid entries with NaN
df.replace(['invalid', 'error', 'NaN', '-9999', '9999'], np.nan, inplace=True)

# Convert features to numeric
independant_val = ['Size', 'Weight', 'Sweetness', 'Crunchiness', 'Juiciness', 'Ripeness', '
df[independant_val] = df[independant_val].apply(pd.to_numeric, errors='coerce')

# Fill NaN values with the median of each column
for col in independant_val:
    df.fillna(df[col].median(), inplace=True)

# Standardize 'Quality' column
df['Quality'] = df['Quality'].str.lower()

df['Quality'] = df['Quality'].map({'good': 1, 'bad': 0})
```

Figure 1: Data Cleaning steps

	A_id	Size	Weight	Sweetness	Crunchiness	Juiciness	Ripeness	Acidity	Quality
0	0	-2.635032	-1.376809	0.436078	1.354921	1.551657	1.252810	2.620943	0.0
1	1	-0.233342	3.440811	-1.195862	1.081771	3.178556	-2.824407	2.012270	1.0
2	2	-4.323396	-0.480859	3.111942	2.471990	-1.279933	-0.833376	5.529057	0.0
3	3	-3.661676	-2.632582	0.828811	0.650855	0.221772	0.742027	3.374035	0.0
4	4	0.393497	-1.417414	-2.961217	2.529450	1.654893	2.960315	-0.388748	0.0
...	...	...	...	...	...	...	...	...	...
4796	4796	-0.518187	-0.518187	-0.518187	-0.518187	-0.518187	-0.518187	-0.518187	NaN
4797	4797	3.099411	-1.691561	-3.679364	0.963606	0.538102	1.660099	-0.544761	1.0
4798	4798	-3.357880	-0.400583	0.694950	0.445515	3.151831	0.675475	2.650962	0.0

Figure 2: Data frame before deleting NaN values in the 'Quality' Column

```
# Drop rows with NaN values in 'Quality' column
df = df.dropna(subset=['Quality'])
```

	A_id	Size	Weight	Sweetness	Crunchiness	Juiciness	Ripeness	Acidity	Quality
0	0	-2.635032	-1.376809	0.436078	1.354921	1.551657	1.252810	2.620943	0.0
1	1	-0.233342	3.440811	-1.195862	1.081771	3.178556	-2.824407	2.012270	1.0
2	2	-4.323396	-0.480859	3.111942	2.471990	-1.279933	-0.833376	5.529057	0.0
3	3	-3.661676	-2.632582	0.828811	0.650855	0.221772	0.742027	3.374035	0.0
4	4	0.393497	-1.417414	-2.961217	2.529450	1.654893	2.960315	-0.388748	0.0
...	...	...	...	...	...	...	...	...	...
4793	4793	-1.571191	1.181532	0.985553	-0.248954	-0.558788	0.147129	-0.450351	1.0
4797	4797	3.099411	-1.691561	-3.679364	0.963606	0.538102	1.660099	-0.544761	1.0
4798	4798	-3.357880	-0.400583	0.694950	0.445515	3.151831	0.675475	2.650962	0.0
4799	4799	-1.382873	-0.276895	0.757917	1.820813	1.344774	0.399316	-0.206760	1.0
4800	4800	-1.572828	-0.816051	-0.165679	0.792687	0.358000	1.662179	-2.748017	1.0

Figure 3: Data Frame after the deletion of NaN value rows in 'Quality' Column

```
# Remove outliers using Z-score
z_scores = np.abs(stats.zscore(df[independant_val]))
outliers = df[(z_scores>3).any(axis=1)]
df = df[(z_scores < 3).all(axis=1)]
print(outliers)
```

	A_id	Size	Weight	Sweetness	Crunchiness	Juiciness	Ripeness	\
139	139	0.440484	0.856487	-0.275917	5.279085	0.014865	-4.071384	
225	225	-0.214485	-2.313936	-3.942249	3.562242	-5.961897	0.301017	
229	229	1.645045	-1.321536	-1.266670	5.567918	-0.702019	-1.996032	
291	291	-4.960451	0.699911	2.862232	-3.674746	-0.028949	0.194217	
355	355	-4.624722	0.256553	-3.986957	-1.721884	0.068654	6.503375	
...	...	...	...	...	...	...	...	
4289	4289	2.334391	-2.464686	-1.738894	6.083439	-0.537013	-1.722726	
4336	4336	-0.922571	-1.977242	3.451115	3.809666	-4.929679	-5.611400	
4463	4463	1.925888	-0.984398	-0.127441	-3.414805	7.364403	-1.342563	
4666	4666	2.594521	-4.535127	0.908736	5.559759	2.374530	2.393040	
4730	4730	0.973114	-1.739375	-1.458617	-3.680468	1.791707	1.019901	

	Acidity	Quality
139	-1.319502	1.0
225	-1.778797	0.0
229	2.245049	1.0
291	-3.726146	1.0
355	-0.918587	0.0
...	...	...
4289	1.801456	1.0
4336	-1.094247	0.0
4463	2.064067	1.0
4666	2.030696	1.0
4730	-1.270915	1.0

[100 rows x 9 columns]

Figure 4: Removal of outliers

II. Data Splitting

The data frame is split into the 2 sets: one for training the model (80%) and the other for testing the data (20%). After splitting the data frame into 'X_train' and 'X_test', they are normalised to scale the features to a standard range.

```
# Split the data into training and test sets
X = df[independant_val]
Y = df['Quality']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Normalize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Figure 5: Data frame split and Normalisation

III. Model Implementation

The K Nearest Neighbour Classification model is used for training and testing the data for the analysis as it is dependent on only distance measurements making it very time efficient. In order to train the model, the `n_neighbours` were selected to be 2 in this analysis (zID: z5511520).

```
# Train the KNN model
knn = KNeighborsClassifier(n_neighbors=2) #zID is 5511520
```

```
knn.fit(X_train, Y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=2)
```

```
# Make predictions and evaluate the model
Y_prediction = knn.predict(X_test)
```

```
print(confusion_matrix(Y_test, Y_prediction))
print(classification_report(Y_test, Y_prediction))
```

```
[[390  20]
 [ 81 331]]
```

	precision	recall	f1-score	support
0.0	0.83	0.95	0.89	410
1.0	0.94	0.80	0.87	412
accuracy			0.88	822
macro avg	0.89	0.88	0.88	822
weighted avg	0.89	0.88	0.88	822

Figure 6: Model Implementation

IV. Model Evaluation

Through the process of training and testing of the data using KNN classifier, we analysed the accuracy of the prediction of the quality of apples which is 88%. That is, the model implemented for this problem can identify 88% of the test cases. In addition, various other metrics were considered to analyse the model's precision which are as follows:

- Precision (Bad) = 0.83, which means 83 out of 100 apples that the model categorised as bad were actually bad and Precision (Good) = 0.94, which means 94 out of 100 apples that the model categorised as good were actually good.
- Recall (Bad) = 0.95, which means the model is effective in ensuring that the poor-quality apples are not mistakenly classified as good and will not reach the customers. But Recall (good) = 0.80, which means that the model could lead a good number of high-quality apples (20%) to be mistaken as bad which may affect the revenue.
- The classification accuracy obtained for the model is 88% which suggests that the model is generally effective in categorising between good and bad apples but this could be improved by effectively processing the data and testing it with another

model as KNN should have its data in all the dimension to be scaled (normalized and standardized) properly.

- The F1 scores for bad and good are 0.89 and 0.87 which indicates that the model is performing well in balancing the precision and recall for both the types.

[[390 20]					
[81 331]]					
		precision	recall	f1-score	support
0.0	0.83	0.95	0.89	410	
1.0	0.94	0.80	0.87	412	
accuracy				0.88	822
macro avg		0.89	0.88	0.88	822
weighted avg		0.89	0.88	0.88	822

Figure 7: Confusion Matrix and Classification Report